



RÉFÉRENCE DE CODE

Plateforme Pâtisserie

Schéma SQL • Structure NestJS • Structure Angular

PARTIE 1 — SCHÉMA SQL COMPLET (Supabase / PostgreSQL)

Copier-coller dans l'éditeur SQL de Supabase (Database → SQL Editor → New query). Exécuter dans l'ordre des sections.

1.1 Extensions & configuration

```
-- Activer UUID v4
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Fonction de mise à jour automatique de updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ language 'plpgsql';
```

1.2 Table users

Extension de auth.users de Supabase. Peuplée automatiquement via un trigger après inscription.

```
CREATE TABLE public.users (
    id          UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    email       TEXT NOT NULL UNIQUE,
    full_name   TEXT NOT NULL,
    phone       TEXT,
    role        TEXT NOT NULL DEFAULT 'customer'
                CHECK (role IN ('customer', 'admin')),
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Trigger updated_at
CREATE TRIGGER set_users_updated_at
BEFORE UPDATE ON public.users
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Trigger : créer le profil dès l'inscription Supabase Auth
CREATE OR REPLACE FUNCTION public.handle_new_user()
```

```

RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.users (id, email, full_name)
    VALUES (NEW.id, NEW.email, COALESCE(NEW.raw_user_meta_data->>'full_name', ''));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

1.3 Table categories

```

CREATE TABLE public.categories (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        TEXT NOT NULL UNIQUE,
    slug        TEXT NOT NULL UNIQUE,
    description TEXT,
    image_url   TEXT,
    sort_order  INTEGER DEFAULT 0,
    is_active   BOOLEAN DEFAULT TRUE,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TRIGGER set_categories_updated_at
BEFORE UPDATE ON public.categories
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Données initiales
INSERT INTO public.categories (name, slug, sort_order) VALUES
    ('Pâtisserie', 'patisserie', 1),
    ('Salé',      'sale',      2),
    ('Livrets',   'livrets',   3);

```

1.4 Table products

```

CREATE TABLE public.products (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id UUID NOT NULL REFERENCES public.categories(id),
    name        TEXT NOT NULL,
    slug        TEXT NOT NULL UNIQUE,
    description TEXT,

```

```

base_price    NUMERIC(10,2) NOT NULL CHECK (base_price >= 0),
image_url     TEXT,
images        TEXT[] DEFAULT '{}',
is_active     BOOLEAN DEFAULT TRUE,
is_featured   BOOLEAN DEFAULT FALSE,
sort_order    INTEGER DEFAULT 0,
created_at    TIMESTAMPTZ DEFAULT NOW(),
updated_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_products_category ON public.products(category_id);
CREATE INDEX idx_products_slug     ON public.products(slug);
CREATE INDEX idx_products_featured ON public.products(is_featured) WHERE
is_featured = TRUE;

CREATE TRIGGER set_products_updated_at
  BEFORE UPDATE ON public.products
  FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

1.5 Table product_options

```

CREATE TABLE public.product_options (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES public.products(id) ON DELETE CASCADE,
  name        TEXT NOT NULL,
  type        TEXT NOT NULL CHECK (type IN ('select', 'text', 'boolean')),
  is_required BOOLEAN DEFAULT FALSE,
  sort_order  INTEGER DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_product_options_product ON public.product_options(product_id);

```

1.6 Table product_option_values

```

CREATE TABLE public.product_option_values (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  option_id   UUID NOT NULL REFERENCES public.product_options(id) ON DELETE
CASCADE,
  value       TEXT NOT NULL,
  price_modifier NUMERIC(10,2) DEFAULT 0,
  sort_order  INTEGER DEFAULT 0
);

```

```
CREATE INDEX idx_option_values_option ON public.product_option_values(option_id);
```

1.7 Table orders

```
CREATE TABLE public.orders (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES public.users(id) ON DELETE SET NULL,
  status            TEXT NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending','confirmed','preparing',
                                   'ready','delivered','cancelled')),
  total_price       NUMERIC(10,2) NOT NULL,
  customer_name     TEXT NOT NULL,
  customer_email    TEXT NOT NULL,
  customer_phone    TEXT,
  delivery_notes    TEXT,
  stripe_payment_intent_id TEXT,
  paid_at           TIMESTAMPTZ,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_orders_user_id ON public.orders(user_id);
CREATE INDEX idx_orders_status  ON public.orders(status);

CREATE TRIGGER set_orders_updated_at
  BEFORE UPDATE ON public.orders
  FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

1.8 Table order_items

```
CREATE TABLE public.order_items (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id          UUID NOT NULL REFERENCES public.orders(id) ON DELETE CASCADE,
  product_id        UUID REFERENCES public.products(id) ON DELETE SET NULL,
  -- Snapshot : indépendant des modifications futures du produit
  product_name      TEXT NOT NULL,
  product_image_url TEXT,
  base_price        NUMERIC(10,2) NOT NULL,
  quantity          INTEGER NOT NULL CHECK (quantity >= 1),
  line_total        NUMERIC(10,2) NOT NULL,
  comment           TEXT
);

CREATE INDEX idx_order_items_order ON public.order_items(order_id);
```

1.9 Table order_item_options

```
CREATE TABLE public.order_item_options (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_item_id UUID NOT NULL REFERENCES public.order_items(id) ON DELETE CASCADE,  
  -- Snapshot des options choisies  
  option_name  TEXT NOT NULL,  
  value        TEXT NOT NULL,  
  price_modifier NUMERIC(10,2) NOT NULL DEFAULT 0  
);  
  
CREATE INDEX idx_order_item_options_item ON  
public.order_item_options(order_item_id);
```

1.10 Row Level Security (RLS)

Activer RLS sur toutes les tables pour que les clients ne voient que leurs propres données.

```
-- Activer RLS  
ALTER TABLE public.users          ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.orders         ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.order_items    ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.order_item_options ENABLE ROW LEVEL SECURITY;  
  
-- users : chaque utilisateur voit uniquement son propre profil  
CREATE POLICY "users_own_row" ON public.users  
  FOR ALL USING (auth.uid() = id);  
  
-- orders : lecture de ses propres commandes  
CREATE POLICY "orders_own" ON public.orders  
  FOR SELECT USING (auth.uid() = user_id);  
  
CREATE POLICY "orders_insert_own" ON public.orders  
  FOR INSERT WITH CHECK (auth.uid() = user_id OR user_id IS NULL);  
  
-- order_items : accessible via la commande parente  
CREATE POLICY "order_items_own" ON public.order_items  
  FOR SELECT USING (  
    EXISTS (SELECT 1 FROM public.orders  
            WHERE id = order_items.order_id AND user_id = auth.uid())  
  );
```

```
-- Produits et catégories : lecture publique
ALTER TABLE public.products  ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.categories ENABLE ROW LEVEL SECURITY;
CREATE POLICY "products_public_read"  ON public.products  FOR SELECT USING
(is_active = TRUE);
CREATE POLICY "categories_public_read" ON public.categories FOR SELECT USING
(is_active = TRUE);
```

PARTIE 2 — STRUCTURE NESTJS (Backend)

Arborescence complète du projet NestJS avec le contenu de chaque fichier clé.

2.1 Arborescence du projet

```
src/  
├─ main.ts  
├─ app.module.ts  
├─ common/  
│   ├── decorators/  
│   │   └─ roles.decorator.ts  
│   └─ guards/  
│       ├── jwt-auth.guard.ts  
│       └─ roles.guard.ts  
├─ modules/  
│   ├── auth/  
│   │   ├── auth.module.ts  
│   │   └─ strategies/  
│   │       └─ jwt.strategy.ts  
│   ├── users/  
│   │   ├── users.module.ts  
│   │   ├── users.controller.ts  
│   │   ├── users.service.ts  
│   │   ├── entities/user.entity.ts  
│   │   └─ dto/  
│   │       └─ update-profile.dto.ts  
│   ├── categories/  
│   │   ├── categories.module.ts  
│   │   ├── categories.controller.ts  
│   │   ├── categories.service.ts  
│   │   ├── entities/category.entity.ts  
│   │   └─ dto/  
│   │       ├── create-category.dto.ts  
│   │       └─ update-category.dto.ts  
│   ├── products/  
│   │   ├── products.module.ts  
│   │   ├── products.controller.ts  
│   │   ├── products.service.ts  
│   │   ├── entities/  
│   │   │   ├── product.entity.ts  
│   │   │   ├── product-option.entity.ts  
│   │   │   └─ product-option-value.entity.ts  
│   │   └─ dto/
```


app.module.ts

```

src/app.module.ts

import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { typeOrmConfig } from '../config/typeorm.config';
import { AuthModule } from '../modules/auth/auth.module';
import { UsersModule } from '../modules/users/users.module';
import { CategoriesModule } from '../modules/categories/categories.module';
import { ProductsModule } from '../modules/products/products.module';
import { OrdersModule } from '../modules/orders/orders.module';
import { StripeModule } from '../modules/stripe/stripe.module';
import { EmailModule } from '../modules/email/email.module';
import { AdminModule } from '../modules/admin/admin.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRootAsync({ useFactory: typeOrmConfig }),
    AuthModule, UsersModule, CategoriesModule,
    ProductsModule, OrdersModule, StripeModule,
    EmailModule, AdminModule,
  ],
})
export class AppModule {}

```

2.3 Module Auth

jwt.strategy.ts

```

src/modules/auth/strategies/jwt.strategy.ts

import { Injectable, UnauthorizedException } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { ConfigService } from '@nestjs/config';
import { createClient } from '@supabase/supabase-js';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(private configService: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: configService.get('SUPABASE_JWT_SECRET'),
    });
  }
}

```

```

    }

    async validate(payload: any) {
      // payload.sub = user UUID de Supabase
      return { id: payload.sub, email: payload.email, role: payload.role };
    }
  }
}

```

roles.guard.ts

```

src/common/guards/roles.guard.ts

import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<string[]>(ROLES_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!requiredRoles) return true;
    const { user } = context.switchToHttp().getRequest();
    return requiredRoles.includes(user?.role);
  }
}

```

2.4 Entités TypeORM

product.entity.ts

```

src/modules/products/entities/product.entity.ts

import { Entity, PrimaryGeneratedColumn, Column, ManyToOne,
  OneToMany, CreateDateColumn, UpdateDateColumn } from 'typeorm';
import { Category } from '../../categories/entities/category.entity';
import { ProductOption } from '../product-option.entity';

@Entity('products')
export class Product {
  @PrimaryGeneratedColumn('uuid')
  id: string;

```

```

@ManyToOne(() => Category, { eager: true })
category: Category;

@Column()
name: string;

@Column({ unique: true })
slug: string;

@Column({ nullable: true })
description: string;

@Column('decimal', { precision: 10, scale: 2 })
basePrice: number;

@Column({ nullable: true })
imageUrl: string;

@Column('text', { array: true, default: '{}' })
images: string[];

@Column({ default: true })
isActive: boolean;

@Column({ default: false })
isFeatured: boolean;

@OneToMany(() => ProductOption, (o) => o.product, { eager: true, cascade:
true })
options: ProductOption[];

@CreateDateColumn()
createdAt: Date;

@UpdateDateColumn()
updatedAt: Date;
}

```

order.entity.ts

```

src/modules/orders/entities/order.entity.ts

import { Entity, PrimaryGeneratedColumn, Column, ManyToOne,
      OneToMany, CreateDateColumn, UpdateDateColumn } from 'typeorm';
import { User } from '../../../users/entities/user.entity';

```

```
import { OrderItem } from './order-item.entity';

export enum OrderStatus {
  PENDING    = 'pending',
  CONFIRMED  = 'confirmed',
  PREPARING  = 'preparing',
  READY      = 'ready',
  DELIVERED  = 'delivered',
  CANCELLED  = 'cancelled',
}

@Entity('orders')
export class Order {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @ManyToOne(() => User, { nullable: true })
  user: User;

  @Column({ type: 'enum', enum: OrderStatus, default: OrderStatus.PENDING })
  status: OrderStatus;

  @Column('decimal', { precision: 10, scale: 2 })
  totalPrice: number;

  @Column()
  customerName: string;

  @Column()
  customerEmail: string;

  @Column({ nullable: true })
  customerPhone: string;

  @Column({ nullable: true })
  deliveryNotes: string;

  @Column({ nullable: true })
  stripePaymentIntentId: string;

  @Column({ type: 'timestampz', nullable: true })
  paidAt: Date;

  @OneToMany(() => OrderItem, (i) => i.order, { cascade: true, eager: true })
  items: OrderItem[];
```

```
@CreateDateColumn()  
createdAt: Date;  
  
@UpdateDateColumn()  
updatedAt: Date;  
}
```

2.5 DTOs principaux

create-order.dto.ts

```
src/modules/orders/dto/create-order.dto.ts  
  
import { IsString, IsEmail, IsArray, IsUUID, IsInt,  
        IsOptional, Min, ValidateNested } from 'class-validator';  
import { Type } from 'class-transformer';  
  
export class SelectedOptionDto {  
    @IsString()  
    optionName: string;  
  
    @IsString()  
    value: string;  
  
    @IsOptional()  
    priceModifier?: number;  
}  
  
export class CreateOrderItemDto {  
    @IsUUID()  
    productId: string;  
  
    @IsInt() @Min(1)  
    quantity: number;  
  
    @IsOptional() @IsString()  
    comment?: string;  
  
    @IsArray()  
    @ValidateNested({ each: true })  
    @Type(() => SelectedOptionDto)  
    selectedOptions: SelectedOptionDto[];  
}  
  
export class CreateOrderDto {
```

```

@isArray()
@ValidateNested({ each: true })
@Type(() => CreateOrderItemDto)
items: CreateOrderItemDto[];

@IsString()
customerName: string;

@IsEmail()
customerEmail: string;

@IsOptional() @IsString()
customerPhone?: string;

@IsOptional() @IsString()
deliveryNotes?: string;
}

```

2.6 Services clés

orders.service.ts — méthode create()

 **src/modules/orders/orders.service.ts** (extrait)

```

async create(dto: CreateOrderDto, userId?: string): Promise<Order> {
  let totalPrice = 0;
  const itemEntities: Partial<OrderItem>[] = [];

  for (const itemDto of dto.items) {
    // 1. Charger le produit depuis la DB (ne jamais faire confiance au client)
    const product = await this.productsRepo.findOne({
      where: { id: itemDto.productId, isActive: true },
      relations: ['options', 'options.values'],
    });
    if (!product) throw new NotFoundException(`Produit ${itemDto.productId} introuvable`);

    // 2. Calculer le total des options
    let optionsTotal = 0;
    const optionSnapshots: Partial<OrderItemOption>[] = [];

    for (const sel of itemDto.selectedOptions) {
      const opt = product.options.find((o) => o.name === sel.optionName);
      const val = opt?.values.find((v) => v.value === sel.value);
      const modifier = val?.priceModifier ?? 0;
      optionsTotal += modifier;
    }
  }
}

```

```
    optionSnapshots.push({ optionName: sel.optionName, value: sel.value,
priceModifier: modifier });
  }

  // 3. Calculer le line total
  const lineTotal = (Number(product.basePrice) + optionsTotal) *
itemDto.quantity;
  totalPrice += lineTotal;

  // 4. Construire le snapshot item
  itemEntities.push({
    productId: product.id,
    productName: product.name,
    productImageUrl: product.imageUrl,
    basePrice: product.basePrice,
    quantity: itemDto.quantity,
    lineTotal,
    comment: itemDto.comment,
    options: optionSnapshots as OrderItemOption[],
  });
}

// 5. Créer la commande en base
const order = this.ordersRepo.create({
  userId,
  customerName: dto.customerName,
  customerEmail: dto.customerEmail,
  customerPhone: dto.customerPhone,
  deliveryNotes: dto.deliveryNotes,
  totalPrice,
  items: itemEntities as OrderItem[],
});

return this.ordersRepo.save(order);
}
```


PARTIE 3 — STRUCTURE ANGULAR (Frontend)

Application Angular 17 avec standalone components, signals et inject().

3.1 Arborescence du projet

```
src/
├─ main.ts
├─ app.config.ts
├─ app.routes.ts
├─ environments/
│   └─ environment.ts
│   └─ environment.prod.ts
├─ app/
│   └─ core/
│       ├── guards/
│       │   ├── auth.guard.ts
│       │   └─ admin.guard.ts
│       ├── interceptors/
│       │   └─ auth.interceptor.ts
│       └─ models/
│           ├── product.model.ts
│           ├── cart.model.ts
│           └─ order.model.ts
│   └─ services/
│       ├── auth.service.ts
│       ├── product.service.ts
│       ├── cart.service.ts
│       ├── order.service.ts
│       ├── stripe.service.ts
│       └─ admin.service.ts
│   └─ shared/
│       ├── components/
│       │   ├── navbar/navbar.component.ts
│       │   ├── footer/footer.component.ts
│       │   └─ cart-drawer/cart-drawer.component.ts
│       └─ pipes/
│           └─ currency-cad.pipe.ts
│   └─ pages/
│       ├── home/home.component.ts
│       └─ products/
│           ├── products.component.ts
│           ├── product-card/product-card.component.ts
│           └─ product-detail/product-detail.component.ts
```

```

|       └─ checkout/checkout.component.ts
|       └─ order-confirmation/order-confirmation.component.ts
|       └─ auth/
|           └─ login/login.component.ts
|           └─ register/register.component.ts
|       └─ dashboard/
|           └─ dashboard-layout.component.ts
|           └─ orders-list/orders-list.component.ts
|           └─ order-detail/order-detail.component.ts
|           └─ profile/profile.component.ts
|       └─ admin/
|           └─ admin-layout.component.ts
|           └─ dashboard/admin-dashboard.component.ts
|           └─ products/admin-products.component.ts
|           └─ categories/admin-categories.component.ts
|           └─ orders/admin-orders.component.ts

```

3.2 Configuration des routes

 **src/app.routes.ts**

```

import { Routes } from '@angular/router';
import { authGuard } from '../app/core/guards/auth.guard';
import { adminGuard } from '../app/core/guards/admin.guard';

export const routes: Routes = [
  { path: '', loadComponent: () =>
import('../app/pages/home/home.component').then(m => m.HomeComponent) },
  { path: 'products', loadComponent: () =>
import('../app/pages/products/products.component').then(m =>
m.ProductsComponent) },
  { path: 'products/:slug', loadComponent: () =>
import('../app/pages/products/product-detail/product-detail.component').then(m =>
m.ProductDetailComponent) },
  { path: 'checkout', loadComponent: () =>
import('../app/pages/checkout/checkout.component').then(m =>
m.CheckoutComponent) },
  { path: 'order-confirmation/:id', loadComponent: () =>
import('../app/pages/order-confirmation/order-confirmation.component').then(m =>
m.OrderConfirmationComponent) },
  { path: 'login', loadComponent: () =>
import('../app/pages/auth/login/login.component').then(m => m.LoginComponent) },
  { path: 'register', loadComponent: () =>
import('../app/pages/auth/register/register.component').then(m =>
m.RegisterComponent) },
  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadComponent: () => import('../app/pages/dashboard/dashboard-
layout.component').then(m => m.DashboardLayoutComponent),

```

```

    children: [
      { path: '', redirectTo: 'orders', pathMatch: 'full' },
      { path: 'orders', loadComponent: () => import('./app/pages/dashboard/orders-list/orders-list.component').then(m => m.OrdersListComponent) },
      { path: 'orders/:id', loadComponent: () =>
import('./app/pages/dashboard/order-detail/order-detail.component').then(m =>
m.OrderDetailComponent) },
      { path: 'profile', loadComponent: () =>
import('./app/pages/dashboard/profile/profile.component').then(m =>
m.ProfileComponent) },
    ]
  },
  {
    path: 'admin',
    canActivate: [adminGuard],
    loadComponent: () => import('./app/pages/admin/admin-layout.component').then(m
=> m.AdminLayoutComponent),
    children: [
      { path: '', redirectTo: 'orders', pathMatch: 'full' },
      { path: 'products', loadComponent: () =>
import('./app/pages/admin/products/admin-products.component').then(m =>
m.AdminProductsComponent) },
      { path: 'categories', loadComponent: () =>
import('./app/pages/admin/categories/admin-categories.component').then(m =>
m.AdminCategoriesComponent) },
      { path: 'orders', loadComponent: () =>
import('./app/pages/admin/orders/admin-orders.component').then(m =>
m.AdminOrdersComponent) },
    ]
  },
  { path: '**', redirectTo: '' }
];

```

3.3 Modèles TypeScript (interfaces)

 **src/app/core/models/product.model.ts**

```

export interface Category {
  id: string;
  name: string;
  slug: string;
  description?: string;
  imageUrl?: string;
  isActive: boolean;
}

export interface ProductOptionValue {
  id: string;
  value: string;
  priceModifier: number;
}

```

```
}

export interface ProductOption {
  id: string;
  name: string;
  type: 'select' | 'text' | 'boolean';
  isRequired: boolean;
  values: ProductOptionValue[];
}

export interface Product {
  id: string;
  name: string;
  slug: string;
  description?: string;
  basePrice: number;
  imageUrl?: string;
  images: string[];
  isFeatured: boolean;
  category: Category;
  options: ProductOption[];
}
```

 **src/app/core/models/cart.model.ts**

```
import { Product } from '../product.model';

export interface SelectedOption {
  optionId: string;
  optionName: string;
  value: string;
  priceModifier: number;
}

export interface CartItem {
  product: Product;
  quantity: number;
  selectedOptions: SelectedOption[];
  comment: string;
  lineTotal: number; // calculé : (basePrice + options) * quantity
}
```

 **src/app/core/models/order.model.ts**

```
export type OrderStatus = 'pending' | 'confirmed' | 'preparing' | 'ready' |
  'delivered' | 'cancelled';
```

```
export interface OrderItemOption {  
  optionName: string;  
  value: string;  
  priceModifier: number;  
}
```

```
export interface OrderItem {  
  id: string;  
  productName: string;  
  productImageUrl?: string;  
  basePrice: number;  
  quantity: number;  
  lineTotal: number;  
  comment?: string;  
  options: OrderItemOption[];  
}
```

```
export interface Order {  
  id: string;  
  status: OrderStatus;  
  totalPrice: number;  
  customerName: string;  
  customerEmail: string;  
  items: OrderItem[];  
  createdAt: string;  
  paidAt?: string;  
}
```

3.4 Services Angular

auth.service.ts

```
 src/app/services/auth.service.ts  
  
import { Injectable, signal, computed } from '@angular/core';  
import { createClient, SupabaseClient, Session, User } from '@supabase/supabase-js';  
import { environment } from '../../environments/environment';  
  
@Injectable({ providedIn: 'root' })  
export class AuthService {  
  private supabase: SupabaseClient;  
  
  // Signals pour état réactif  
  session = signal<Session | null>(null);  
  user = computed(() => this.session()?.user ?? null);  
}
```

```

isLoggedIn = computed(() => !!this.session());
isAdmin = computed(() => this.user()?.user_metadata['role'] === 'admin');

constructor() {
  this.supabase = createClient(environment.supabaseUrl,
environment.supabaseAnonKey);

  // Initialiser la session au chargement
  this.supabase.auth.getSession().then(({ data }) => {
    this.session.set(data.session);
  });

  // Écouter les changements de session
  this.supabase.auth.onAuthStateChange(_, session) => {
    this.session.set(session);
  });
}

async signUp(email: string, password: string, fullName: string) {
  return this.supabase.auth.signUp({
    email, password,
    options: { data: { full_name: fullName } }
  });
}

async signIn(email: string, password: string) {
  return this.supabase.auth.signInWithPassword({ email, password });
}

async signOut() {
  await this.supabase.auth.signOut();
  this.session.set(null);
}

getAccessToken(): string | null {
  return this.session()?.access_token ?? null;
}
}

```

cart.service.ts

```

src/app/services/cart.service.ts

import { Injectable, signal, computed } from '@angular/core';
import { CartItem, SelectedOption } from '../core/models/cart.model';
import { Product } from '../core/models/product.model';

```

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private STORAGE_KEY = 'patisserie_cart';

  // Charger depuis localStorage au démarrage
  items = signal<CartItem[]>(this.loadFromStorage());

  total = computed(() =>
    this.items().reduce((sum, item) => sum + item.lineTotal, 0)
  );

  itemCount = computed(() =>
    this.items().reduce((sum, item) => sum + item.quantity, 0)
  );

  addItem(product: Product, quantity: number, selectedOptions: SelectedOption[],
    comment: string) {
    const lineTotal = this.computeLineTotal(product.basePrice, selectedOptions,
      quantity);
    const newItem: CartItem = { product, quantity, selectedOptions, comment,
      lineTotal };
    this.items.update((items) => [...items, newItem]);
    this.saveToStorage();
  }

  updateQuantity(index: number, quantity: number) {
    this.items.update((items) => {
      const updated = [...items];
      updated[index] = {
        ...updated[index],
        quantity,
        lineTotal: this.computeLineTotal(updated[index].product.basePrice,
          updated[index].selectedOptions, quantity)
      };
      return updated;
    });
    this.saveToStorage();
  }

  removeItem(index: number) {
    this.items.update((items) => items.filter((_, i) => i !== index));
    this.saveToStorage();
  }

  clearCart() {
    this.items.set([]);
  }
}
```

```

    localStorage.removeItem(this.STORAGE_KEY);
  }

  private computeLineTotal(basePrice: number, options: SelectedOption[], qty:
number): number {
    const optionsTotal = options.reduce((s, o) => s + o.priceModifier, 0);
    return (basePrice + optionsTotal) * qty;
  }

  private saveToStorage() {
    localStorage.setItem(this.STORAGE_KEY, JSON.stringify(this.items()));
  }

  private loadFromStorage(): CartItem[] {
    try {
      const raw = localStorage.getItem(this.STORAGE_KEY);
      return raw ? JSON.parse(raw) : [];
    } catch { return []; }
  }
}

```

auth.interceptor.ts

```

src/app/core/interceptors/auth.interceptor.ts

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.getAccessToken();

  if (token) {
    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
    return next(authReq);
  }

  return next(req);
};

```

product.service.ts

```

src/app/services/product.service.ts

```



```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { shareReplay } from 'rxjs/operators';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { Product, Category } from '../../core/models/product.model';

export interface ProductFilters {
  category?: string;
  featured?: boolean;
  search?: string;
  page?: number;
  limit?: number;
}

export interface PaginatedProducts {
  data: Product[];
  total: number;
  page: number;
  limit: number;
}

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
  private base = `${environment.apiUrl}/api`;

  // Cache des catégories (ne changent pas souvent)
  categories$ = this.http.get<Category[]>(`${this.base}/categories`).pipe(
    shareReplay(1)
  );

  getProducts(filters: ProductFilters = {}): Observable<PaginatedProducts> {
    let params = new HttpParams();
    if (filters.category) params = params.set('category', filters.category);
    if (filters.featured) params = params.set('featured', 'true');
    if (filters.search) params = params.set('search', filters.search);
    if (filters.page) params = params.set('page', filters.page.toString());
    if (filters.limit) params = params.set('limit', filters.limit.toString());
    return this.http.get<PaginatedProducts>(`${this.base}/products`, { params });
  }

  getProductBySlug(slug: string): Observable<Product> {
    return this.http.get<Product>(`${this.base}/products/${slug}`);
  }
}
```

3.5 Guards

auth.guard.ts

```
src/app/core/guards/auth.guard.ts

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) return true;
  return router.createUrlTree(['/login']);
};

// admin.guard.ts
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isAdmin()) return true;
  return router.createUrlTree(['/']);
};
```

3.6 Configuration applicative

```
src/app.config.ts

import { ApplicationConfig } from '@angular/core';
import { provideRouter, withViewTransitions } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './app/core/interceptors/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withViewTransitions()),
    provideHttpClient(withInterceptors([authInterceptor])),
  ]
};
```

```
src/environments/environment.ts
```

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:3000',  
  supabaseUrl: 'https://xxxx.supabase.co',  
  supabaseAnonKey: 'YOUR_ANON_KEY',  
  stripePublishableKey: 'pk_test_...',  
};
```

PARTIE 2 (suite) — Contrôleurs & Services NestJS

2.7 products.controller.ts

```

src/modules/products/products.controller.ts

import { Controller, Get, Post, Patch, Delete,
    Param, Query, Body, UseGuards } from '@nestjs/common';
import { ProductsService } from '../products.service';
import { CreateProductDto } from '../dto/create-product.dto';
import { CreateOptionDto } from '../dto/create-option.dto';
import { CreateOptionValueDto } from '../dto/create-option-value.dto';
import { JwtAuthGuard } from '../../../common/guards/jwt-auth.guard';
import { RolesGuard } from '../../../common/guards/roles.guard';
import { Roles } from '../../../common/decorators/roles.decorator';

// Routes publiques : GET /api/products
@Controller('products')
export class ProductsController {
    constructor(private readonly productsService: ProductsService) {}

    @Get()
    findAll(@Query() query: {
        category?: string;
        featured?: string;
        search?: string;
        page?: string;
        limit?: string;
    }) {
        return this.productsService.findAll({
            category: query.category,
            featured: query.featured === 'true',
            search: query.search,
            page: query.page ? parseInt(query.page) : 1,
            limit: query.limit ? parseInt(query.limit) : 12,
        });
    }

    @Get('/:slug')
    findOne(@Param('slug') slug: string) {
        return this.productsService.findBySlug(slug);
    }
}

// Routes admin : POST/PATCH/DELETE /api/admin/products

```

```
@Controller('admin/products')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('admin')
export class AdminProductsController {
  constructor(private readonly productsService: ProductsService) {}

  @Post()
  create(@Body() dto: CreateProductDto) {
    return this.productsService.create(dto);
  }

  @Patch('/:id')
  update(@Param('id') id: string, @Body() dto: Partial<CreateProductDto>) {
    return this.productsService.update(id, dto);
  }

  @Delete('/:id')
  remove(@Param('id') id: string) {
    return this.productsService.softDelete(id);
  }

  // Options
  @Post('/:productId/options')
  addOption(@Param('productId') productId: string, @Body() dto: CreateOptionDto) {
    return this.productsService.addOption(productId, dto);
  }

  @Delete('options/:optionId')
  removeOption(@Param('optionId') optionId: string) {
    return this.productsService.removeOption(optionId);
  }

  // Valeurs d'options
  @Post('options/:optionId/values')
  addOptionValue(@Param('optionId') optionId: string, @Body() dto:
CreateOptionValueDto) {
    return this.productsService.addOptionValue(optionId, dto);
  }

  @Patch('values/:valueId')
  updateOptionValue(@Param('valueId') valueId: string, @Body() dto:
Partial<CreateOptionValueDto>) {
    return this.productsService.updateOptionValue(valueId, dto);
  }

  @Delete('values/:valueId')
  removeOptionValue(@Param('valueId') valueId: string) {
```

```

    return this.productsService.removeOptionValue(valueId);
  }
}

```

2.8 products.service.ts

```

src/modules/products/products.service.ts

import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Product } from '../entities/product.entity';
import { ProductOption } from '../entities/product-option.entity';
import { ProductOptionValue } from '../entities/product-option-value.entity';

@Injectable()
export class ProductsService {
  constructor(
    @InjectRepository(Product) private productsRepo:
    Repository<Product>,
    @InjectRepository(ProductOption) private optionsRepo:
    Repository<ProductOption>,
    @InjectRepository(ProductOptionValue) private valuesRepo:
    Repository<ProductOptionValue>,
  ) {}

  async findAll(filters: { category?: string; featured?: boolean; search?: string;
    page: number; limit: number; }) {
    const qb = this.productsRepo.createQueryBuilder('product')
      .leftJoinAndSelect('product.category', 'category')
      .leftJoinAndSelect('product.options', 'options')
      .leftJoinAndSelect('options.values', 'values')
      .where('product.isActive = :isActive', { isActive: true });

    if (filters.category) qb.andWhere('category.slug = :slug', { slug:
    filters.category });
    if (filters.featured) qb.andWhere('product.isFeatured = true');
    if (filters.search) qb.andWhere(
      'LOWER(product.name) LIKE :search OR LOWER(product.description)
    LIKE :search',
      { search: `%${filters.search.toLowerCase()}%` }
    );

    const total = await qb.getCount();
    const data = await qb
      .orderBy('product.sortOrder', 'ASC')
      .skip((filters.page - 1) * filters.limit)
      .take(filters.limit)
  }
}

```

```
.getMany();

return { data, total, page: filters.page, limit: filters.limit };
}

async findBySlug(slug: string): Promise<Product> {
  const product = await this.productsRepo.findOne({ where: { slug, isActive:
true } });
  if (!product) throw new NotFoundException('Produit introuvable');
  return product;
}

async create(dto: any): Promise<Product> {
  const slug = dto.name.toLowerCase().replace(/^[a-z0-9]+/g, '-');
  const product = this.productsRepo.create({ ...dto, slug });
  return this.productsRepo.save(product);
}

async update(id: string, dto: any): Promise<Product> {
  await this.productsRepo.update(id, dto);
  return this.productsRepo.findOneOrFail({ where: { id } });
}

async softDelete(id: string) {
  return this.productsRepo.update(id, { isActive: false });
}

async addOption(productId: string, dto: any): Promise<ProductOption> {
  const option = this.optionsRepo.create({ ...dto, productId });
  return this.optionsRepo.save(option);
}

async removeOption(optionId: string) {
  return this.optionsRepo.delete(optionId);
}

async addOptionValue(optionId: string, dto: any): Promise<ProductOptionValue> {
  const val = this.valuesRepo.create({ ...dto, optionId });
  return this.valuesRepo.save(val);
}

async updateOptionValue(valueId: string, dto: any) {
  return this.valuesRepo.update(valueId, dto);
}

async removeOptionValue(valueId: string) {
```

```

    return this.valuesRepo.delete(valueId);
  }
}

```

2.9 orders.controller.ts

```

src/modules/orders/orders.controller.ts
● ● ●
import { Controller, Post, Get, Patch, Param, Body,
    UseGuards, Request } from '@nestjs/common';
import { OrdersService } from '../orders.service';
import { CreateOrderDto } from '../dto/create-order.dto';
import { UpdateOrderStatusDto } from '../dto/update-order-status.dto';
import { JwtAuthGuard } from '../../common/guards/jwt-auth.guard';
import { RolesGuard } from '../../common/guards/roles.guard';
import { Roles } from '../../common/decorators/roles.decorator';

// Créer une commande (avec ou sans compte)
@Controller('orders')
export class OrdersController {
  constructor(private readonly ordersService: OrdersService) {}

  // POST /api/orders/create-intent
  @Post('create-intent')
  createIntent(@Body() dto: CreateOrderDto, @Request() req: any) {
    const userId = req.user?.id; // undefined si invité
    return this.ordersService.createWithPaymentIntent(dto, userId);
  }

  // GET /api/orders/mine - commandes du client connecté
  @Get('mine')
  @UseGuards(JwtAuthGuard)
  findMine(@Request() req: any) {
    return this.ordersService.findByUser(req.user.id);
  }

  // GET /api/orders/mine/:id
  @Get('mine/:id')
  @UseGuards(JwtAuthGuard)
  findOne(@Param('id') id: string, @Request() req: any) {
    return this.ordersService.findOneForUser(id, req.user.id);
  }
}

// Routes admin
@Controller('admin/orders')

```



```

@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('admin')
export class AdminOrdersController {
  constructor(private readonly ordersService: OrdersService) {}

  @Get()
  findAll(@Request() req: any) {
    const status = req.query?.status;
    return this.ordersService.findAll(status);
  }

  @Patch('/:id/status')
  updateStatus(@Param('id') id: string, @Body() dto: UpdateOrderStatusDto) {
    return this.ordersService.updateStatus(id, dto.status);
  }
}

```

2.10 stripe.service.ts + stripe.controller.ts

```

src/modules/stripe/stripe.service.ts

import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import Stripe from 'stripe';

@Injectable()
export class StripeService {
  private stripe: Stripe;

  constructor(private config: ConfigService) {
    this.stripe = new Stripe(this.config.get('STRIPE_SECRET_KEY'), {
      apiVersion: '2024-04-10',
    });
  }

  async createPaymentIntent(amount: number, orderId: string):
    Promise<Stripe.PaymentIntent> {
    return this.stripe.paymentIntents.create({
      amount: Math.round(amount * 100), // en centimes
      currency: 'cad',
      metadata: { orderId },
    });
  }

  constructWebhookEvent(payload: Buffer, signature: string): Stripe.Event {
    return this.stripe.webhooks.constructEvent(

```

```

        payload,
        signature,
        this.config.get('STRIPE_WEBHOOK_SECRET'),
    );
}
}

```

src/modules/stripe/stripe.controller.ts

```

import { Controller, Post, Body, Headers, RawBodyRequest, Req } from
 '@nestjs/common';
import { StripeService } from '../stripe.service';
import { OrdersService } from '../orders/orders.service';
import { EmailService } from '../email/email.service';
import { Request } from 'express';

@Controller('stripe')
export class StripeController {
    constructor(
        private stripeService: StripeService,
        private ordersService: OrdersService,
        private emailService: EmailService,
    ) {}

    // POST /api/stripe/webhook
    // Enregistrer dans Stripe Dashboard : https://votre-backend/api/stripe/webhook
    @Post('webhook')
    async handleWebhook(
        @Req() req: RawBodyRequest<Request>,
        @Headers('stripe-signature') signature: string,
    ) {
        const event = this.stripeService.constructWebhookEvent(req.rawBody,
            signature);

        if (event.type === 'payment_intent.succeeded') {
            const intent = event.data.object as any;
            const orderId = intent.metadata.orderId;

            // 1. Mettre à jour la commande
            const order = await this.ordersService.markAsPaid(orderId, intent.id);

            // 2. Envoyer l'email de confirmation
            await this.emailService.sendOrderConfirmation(order);
        }

        return { received: true };
    }
}

```

```

}
}

```

2.11 email.service.ts

 src/modules/email/email.service.ts

```

import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { Resend } from 'resend';
import { Order } from '../../orders/entities/order.entity';

@Injectable()
export class EmailService {
  private resend: Resend;

  constructor(private config: ConfigService) {
    this.resend = new Resend(this.config.get('RESEND_API_KEY'));
  }

  async sendOrderConfirmation(order: Order) {
    const itemsHtml = order.items.map(item => `
      <tr>
        <td>${item.productName}</td>
        <td>${item.quantity}</td>
        <td>${item.lineTotal.toFixed(2)} $</td>
      </tr>
    `).join('');

    await this.resend.emails.send({
      from: 'Pâtisserie <noreply@votredomaine.com>',
      to: order.customerEmail,
      subject: `✅ Commande #${order.id.slice(0, 8)} confirmée`,
      html: `
        <h2>Merci pour votre commande, ${order.customerName} !</h2>
        <table>${itemsHtml}</table>
        <p><strong>Total : ${order.totalPrice.toFixed(2)} $</strong></p>
      `,
    });
  }

  async sendStatusUpdate(order: Order) {
    const statusLabels: Record<string, string> = {
      confirmed: 'confirmée',
      preparing: 'en préparation',
      ready: 'prête à récupérer',
    };
  }
}

```

```
    delivered: 'livrée',  
    cancelled: 'annulée',  
  };  
  
  await this.resend.emails.send({  
    from: 'Pâtisserie <noreply@votredomaine.com>',  
    to: order.customerEmail,  
    subject: `📦 Votre commande est ${statusLabels[order.status] ??  
order.status}`,  
    html: `

Bonjour ${order.customerName}, votre commande est désormais  
<strong>${statusLabels[order.status]}</strong>.</p>`,  
  });  
}  
}


```

PARTIE 3 (suite) — Composants Angular clés

3.7 order.service.ts

 `src/app/services/order.service.ts`

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { Order } from '../../core/models/order.model';
import { CartItem } from '../../core/models/cart.model';

@Injectable({ providedIn: 'root' })
export class OrderService {
  private http = inject(HttpClient);
  private base = `${environment.apiUrl}/api`;

  createIntent(items: CartItem[], customerInfo: {
    customerName: string;
    customerEmail: string;
    customerPhone?: string;
    deliveryNotes?: string;
  }): Observable<{ orderId: string; clientSecret: string }> {
    const payload = {
      ...customerInfo,
      items: items.map(item => ({
        productId: item.product.id,
        quantity: item.quantity,
        comment: item.comment,
        selectedOptions: item.selectedOptions.map(o => ({
          optionName: o.optionName,
          value: o.value,
        })),
      })),
    };
    return this.http.post<{ orderId: string; clientSecret: string }>(`${this.base}/orders/create-intent`, payload);
  }

  getMyOrders(): Observable<Order[]> {
    return this.http.get<Order[]>(`${this.base}/orders/mine`);
  }

  getOrderById(id: string): Observable<Order> {
```

```

    return this.http.get<Order>(`${this.base}/orders/mine/${id}`);
  }
}

```

3.8 stripe.service.ts (Angular)

 **src/app/services/stripe.service.ts**

```

import { Injectable } from '@angular/core';
import { loadStripe, Stripe, StripeCardElement } from '@stripe/stripe-js';
import { environment } from '../../environments/environment';

@Injectable({ providedIn: 'root' })
export class StripeService {
  private stripePromise = loadStripe(environment.stripePublishableKey);
  private cardElement: StripeCardElement | null = null;

  async mountCardElement(selector: string): Promise<void> {
    const stripe = await this.stripePromise;
    if (!stripe) return;
    const elements = stripe.elements();
    this.cardElement = elements.create('card', {
      style: { base: { fontSize: '16px', color: '#1a1a1a' } }
    });
    this.cardElement.mount(selector);
  }

  async confirmPayment(clientSecret: string): Promise<{ error?: string }> {
    const stripe = await this.stripePromise;
    if (!stripe || !this.cardElement) return { error: 'Stripe non initialisé' };

    const { error, paymentIntent } = await stripe.confirmCardPayment(clientSecret, {
      payment_method: { card: this.cardElement },
    });

    if (error) return { error: error.message };
    if (paymentIntent?.status === 'succeeded') return {};
    return { error: 'Paielement non complété' };
  }

  destroyCardElement() {
    this.cardElement?.destroy();
    this.cardElement = null;
  }
}

```

3.9 checkout.component.ts

 `src/app/pages/checkout/checkout.component.ts`

```
import { Component, OnInit, OnDestroy, inject, signal } from '@angular/core';
import { FormBuilder, Validators, ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { CartService } from '../../services/cart.service';
import { OrderService } from '../../services/order.service';
import { StripeService } from '../../services/stripe.service';

@Component({
  selector: 'app-checkout',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  template: `
    <div class="checkout-container">
      <h1>Finaliser la commande</h1>

      <!-- Récapitulatif panier -->
      <section class="cart-summary">
        <div *ngFor="let item of cart.items()" class="cart-item">
          <span>{{ item.product.name }} × {{ item.quantity }}</span>
          <span>{{ item.lineTotal | currency:'CAD':'symbol-narrow' }}</span>
        </div>
        <div class="total"><strong>Total : {{ cart.total() |
currency:'CAD':'symbol-narrow' }}</strong></div>
      </section>

      <!-- Formulaire client -->
      <form [formGroup]="form">
        <input formControlName="customerName" placeholder="Nom complet *" />
        <input formControlName="customerEmail" placeholder="Email *"
type="email" />
        <input formControlName="customerPhone" placeholder="Téléphone" />
        <textarea formControlName="deliveryNotes" placeholder="Instructions
spéciales..."></textarea>
      </form>

      <!-- Carte Stripe -->
      <div id="card-element"></div>

      <!-- Erreur -->
      <p *ngIf="error()" class="error">{{ error() }}</p>
    </div>`
})
export class CheckoutComponent {
  private router = inject(Router);
  private cartService = inject(CartService);
  private orderService = inject(OrderService);
  private stripeService = inject(StripeService);

  form = this.fb.group({
    customerName: ['', Validators.required],
    customerEmail: ['', Validators.required],
    customerPhone: ['', Validators.required],
    deliveryNotes: ''
  });

  error = signal('');

  ngOnInit(): void {
    this.cartService.getCart().subscribe((cart) => {
      this.cartService.setCart(cart);
    });
  }

  ngOnDestroy(): void {
    this.cartService.destroy();
  }

  checkout(): void {
    this.orderService.createOrder().subscribe((order) => {
      this.router.navigate(['/confirmation']);
    });
  }
}
```

```

    <!-- Bouton payer -->
    <button (click)="pay()" [disabled]="loading() || form.invalid">
      {{ loading() ? 'Traitement...' : 'Payer ' + (cart.total() |
currency:'CAD':'symbol-narrow') }}
    </button>
  </div>
  `
  })
  export class CheckoutComponent implements OnInit, OnDestroy {
    cart      = inject(CartService);
    order      = inject(OrderService);
    stripe     = inject(StripeService);
    router     = inject(Router);
    fb         = inject(FormBuilder);

    loading = signal(false);
    error   = signal<string | null>(null);

    form = this.fb.group({
      customerName: ['', Validators.required],
      customerEmail: ['', [Validators.required, Validators.email]],
      customerPhone: ['', ],
      deliveryNotes: ['', ],
    });

    async ngOnInit() {
      await this.stripe.mountCardElement('#card-element');
    }

    ngOnDestroy() { this.stripe.destroyCardElement(); }

    async pay() {
      if (this.form.invalid) return;
      this.loading.set(true);
      this.error.set(null);

      // 1. Créer la commande côté backend + obtenir clientSecret
      this.order.createIntent(this.cart.items(), this.form.value as any).subscribe({
        next: async ({ orderId, clientSecret }) => {
          // 2. Confirmer le paiement Stripe
          const result = await this.stripe.confirmPayment(clientSecret);
          if (result.error) {
            this.error.set(result.error);
            this.loading.set(false);
            return;
          }
        }
      })
      // 3. Succès → vider le panier et rediriger

```



```

        this.cart.clearCart();
        this.router.navigate(['/order-confirmation', orderId]);
    },
    error: (err) => {
        this.error.set('Erreur lors de la création de la commande.');
```

```

        this.loading.set(false);
    }
});
}
}
```

3.10 product-detail.component.ts

 **src/app/pages/products/product-detail/product-detail.component.ts**

```

import { Component, OnInit, inject, signal, computed } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ProductService } from '../../../services/product.service';
import { CartService } from '../../../services/cart.service';
import { Product, ProductOption } from '../../../core/models/product.model';
import { SelectedOption } from '../../../core/models/cart.model';
```

```

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <div *ngIf="product()" class="product-detail">
      <img [src]="product()!.imageUrl" [alt]="product()!.name" />

      <div class="product-info">
        <h1>{{ product()!.name }}</h1>
        <p>{{ product()!.description }}</p>

        <!-- Options de personnalisation -->
        <div *ngFor="let option of product()!.options" class="option-group">
          <label><strong>{{ option.name }}</strong>
            <span *ngIf="option.isRequired" class="required">*</span>
          </label>

          <!-- Type select -->
          <div *ngIf="option.type === 'select'" class="option-values">
            <label *ngFor="let val of option.values" class="option-radio">
              <input type="radio" [name]="option.id"

```

```

        (change)="selectOption(option, val.value, val.priceModifier)" />
        {{ val.value }}
        <span *ngIf="val.priceModifier !== 0" class="modifier">
            {{ val.priceModifier > 0 ? '+' : '-' }}{{ val.priceModifier |
currency:'CAD':'symbol-narrow' }}
        </span>
    </label>
</div>

<!-- Type text -->
<input *ngIf="option.type === 'text'" type="text"
    [placeholder]='Précisez...' maxlength="200"
    (input)="selectOption(option, $any($event.target).value, 0)" />

<!-- Type boolean -->
<input *ngIf="option.type === 'boolean'" type="checkbox"
    (change)="toggleBoolean(option, $any($event.target).checked)" />
</div>

<!-- Commentaire libre -->
<textarea [(ngModel)]="comment" placeholder="Instructions spéciales,
allergies, message..." maxlength="500"></textarea>

<!-- Quantité -->
<div class="quantity">
    <button (click)="qty.set(qty() - 1)" [disabled]="qty() <= 1">-</button>
    <span>{{ qty() }}</span>
    <button (click)="qty.set(qty() + 1)">+</button>
</div>

<!-- Prix dynamique -->
<div class="price">
    <span class="unit-price">{{ computedUnitPrice() |
currency:'CAD':'symbol-narrow' }} / unité</span>
    <span class="total-price">Total : {{ computedTotal() |
currency:'CAD':'symbol-narrow' }}</span>
</div>

    <button (click)="addToCart()" [disabled]="!canAdd()">Ajouter au
panier</button>
</div>
</div>
,
})
export class ProductDetailComponent implements OnInit {
    route = inject(ActivatedRoute);
    router = inject(Router);
    productSvc = inject(ProductService);

```

```

cartSvc      = inject(CartService);

product      = signal<Product | null>(null);
selected     = signal<SelectedOption[]>([]);
qty          = signal(1);
comment      = '';

computedUnitPrice = computed(() => {
  const base = this.product()?.basePrice ?? 0;
  const opts = this.selected().reduce((s, o) => s + o.priceModifier, 0);
  return base + opts;
});

computedTotal = computed(() => this.computedUnitPrice() * this.qty());

canAdd = computed(() => {
  const required = this.product()?.options.filter(o => o.isRequired) ?? [];
  return required.every(o => this.selected().some(s => s.optionName ===
o.name));
});

ngOnInit() {
  const slug = this.route.snapshot.paramMap.get('slug')!;
  this.productSvc.getProductBySlug(slug).subscribe({
    next: (p) => this.product.set(p),
    error: () => this.router.navigate(['/products']),
  });
}

selectOption(option: ProductOption, value: string, priceModifier: number) {
  this.selected.update((sel) => [
    ...sel.filter(s => s.optionName !== option.name),
    { optionId: option.id, optionName: option.name, value, priceModifier },
  ]);
}

toggleBoolean(option: ProductOption, checked: boolean) {
  const modifier = option.values[0]?.priceModifier ?? 0;
  this.selectOption(option, checked ? 'Oui' : 'Non', checked ? modifier : 0);
}

addToCart() {
  this.cartSvc.addItem(this.product()!, this.qty(), this.selected(),
this.comment);
  this.router.navigate(['/products']);
}
}

```

3.11 cart-drawer.component.ts

 src/app/shared/components/cart-drawer/cart-drawer.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Router } from '@angular/router';
import { CartService } from '../../../services/cart.service';

@Component({
  selector: 'app-cart-drawer',
  standalone: true,
  imports: [CommonModule],
  template: `
    <!-- Overlay -->
    <div class="overlay" *ngIf="open()" (click)="close()"></div>

    <!-- Drawer -->
    <div class="drawer" [class.open]="open()">
      <div class="drawer-header">
        <h2>Votre panier ({{ cart.itemCount() }})</h2>
        <button (click)="close()">X</button>
      </div>

      <div class="drawer-body">
        <div *ngIf="cart.items().length === 0" class="empty">
          Votre panier est vide
        </div>

        <div *ngFor="let item of cart.items(); let i = index" class="cart-item">
          <img [src]="item.product.imageUrl" />
          <div class="item-info">
            <strong>{{ item.product.name }}</strong>
            <small *ngFor="let opt of item.selectedOptions">
              {{ opt.optionName }} : {{ opt.value }}
            </small>
            <small *ngIf="item.comment">📝 {{ item.comment }}</small>
          </div>
          <div class="item-qty">
            <button (click)="cart.updateQuantity(i, item.quantity - 1)"
[disabled]="item.quantity <= 1">-</button>
            {{ item.quantity }}
            <button (click)="cart.updateQuantity(i, item.quantity + 1)">+</button>
          </div>
          <span>{{ item.lineTotal | currency:'CAD':'symbol-narrow' }}</span>
        </div>
      </div>
    </div>`
})
export class CartDrawerComponent {
  private router = inject(Router);
  private cartService = inject(CartService);
  open = signal(false);
  close() {
    this.open.set(false);
  }
}
```

```

        <button (click)="cart.removeItem(i)" class="remove">🗑️</button>
      </div>
    </div>

    <div class="drawer-footer" *ngIf="cart.items().length > 0">
      <div class="total">Total : <strong>{{ cart.total() |
currency:'CAD':'symbol-narrow' }}</strong></div>
      <button (click)="goToCheckout()" class="btn-primary">Passer la
commande</button>
    </div>
  </div>
</div>
`
  })
}
export class CartDrawerComponent {
  cart = inject(CartService);
  router = inject(Router);
  open = signal(false);

  toggle() { this.open.update(v => !v); }
  close() { this.open.set(false); }

  goToCheckout() {
    this.close();
    this.router.navigate(['/checkout']);
  }
}

```

3.12 admin.service.ts

```

src/app/services/admin.service.ts

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { Product, Category, ProductOption, ProductOptionValue } from
'../core/models/product.model';
import { Order } from '../core/models/order.model';

@Injectable({ providedIn: 'root' })
export class AdminService {
  private http = inject(HttpClient);
  private base = `${environment.apiUrl}/api/admin`;

  // — Produits —————
  createProduct(data: Partial<Product>): Observable<Product> {
    return this.http.post<Product>(`${this.base}/products`, data);
  }
}

```

```

    }

    updateProduct(id: string, data: Partial<Product>): Observable<Product> {
        return this.http.patch<Product>(`${this.base}/products/${id}`, data);
    }

    deleteProduct(id: string): Observable<void> {
        return this.http.delete<void>(`${this.base}/products/${id}`);
    }

    // — Options —————
    addOption(productId: string, data: Partial<ProductOption>):
    Observable<ProductOption> {
        return this.http.post<ProductOption>(`${this.base}/products/${productId}/
options`, data);
    }

    deleteOption(optionId: string): Observable<void> {
        return this.http.delete<void>(`${this.base}/products/options/${optionId}`);
    }

    // — Valeurs d'options —————
    addOptionValue(optionId: string, data: Partial<ProductOptionValue>):
    Observable<ProductOptionValue> {
        return this.http.post<ProductOptionValue>(`${this.base}/products/options/${
optionId}/values`, data);
    }

    deleteOptionValue(valueId: string): Observable<void> {
        return this.http.delete<void>(`${this.base}/products/values/${valueId}`);
    }

    // — Catégories —————
    createCategory(data: Partial<Category>): Observable<Category> {
        return this.http.post<Category>(`${this.base}/categories`, data);
    }

    updateCategory(id: string, data: Partial<Category>): Observable<Category> {
        return this.http.patch<Category>(`${this.base}/categories/${id}`, data);
    }

    deleteCategory(id: string): Observable<void> {
        return this.http.delete<void>(`${this.base}/categories/${id}`);
    }

    // — Commandes —————
    getOrders(status?: string): Observable<Order[]> {
        const params = status ? `?status=${status}` : '';
        return this.http.get<Order[]>(`${this.base}/orders${params}`);
    }

    updateOrderStatus(id: string, status: string): Observable<Order> {
        return this.http.patch<Order>(`${this.base}/orders/${id}/status`, { status });
    }
}

```

✦ Référence de code v1.0 • Plateforme Pâtisserie ✦